Module 1
INTRODUCTION

Syllabus:

Introduction: Notion of algorithm, Fundamentals of Algorithmic Problem Solving, Fundamentals of the Analysis of Algorithmic Efficiency: Analysis frame work, Asymptotic Notations and Basic Efficiency Classes, Mathematical Analysis of Non-recursive and Recursive Algorithms. Brute Force: Selection Sort and Bubble Sort.

Q. What is an algorithm? What are the properties of an algorithm? Explain with an example. ***(8 M)***

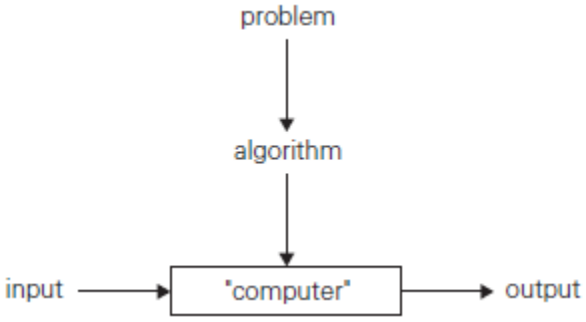
Algorithm:

An algorithm is a finite sequence of unambiguous instructions for solving a problem i.e, for obtaining a required output for any legitimate input in a finite amount of time.

Several important points to be noted:

1. The non ambiguity requirement for each step of an algorithm cannot be compromised.
2. The range of inputs for which an algorithm works has to be specified carefully.
3. The same algorithm can be represented in several different ways.
4. Several algorithms for solving the same problem may exist.
5. Algorithms for the same problem can be based on very different ideas and can solve the problem with dramatically different speeds.

Notion of algorithm



Properties of an algorithm

1. **Input:** Each algorithm should have zero or more inputs.
2. **Output:** The algorithm should produce correct results. At least one output has to be produced.
3. **Definiteness:** Each instruction should be clear and unambiguous.

4. **Effectiveness:** The instructions should be simple and should transform the given input to the desired output.
5. **Finiteness:** The algorithm must terminate after a finite sequence of instructions.

Example 1: Algorithm *Euclid (m,n)*

//Computes gcd*(m, n)* by Euclid's algorithm

//Input: Two non-negative, not-both-zero integers *m* and *n*

//Output: Greatest common divisor of *m* and *n*

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Example 2: Algorithm *Sieve (n)*

//Implements the sieve of Eratosthenes

//Input: A positive integer $n > 1$

//Output: Array *L* of all prime numbers less than or equal to *n*

for $p \leftarrow 2$ **to** *n* **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\sqrt{n}$ **do** //see note before pseudocode

if $A[p] \neq 0$ //*p* hasn't been eliminated on previous passes

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ //mark element as eliminated

$j \leftarrow j + p$

//copy the remaining elements of *A* to array *L* of the primes

$i \leftarrow 0$

for $p \leftarrow 2$ **to** *n* **do**

if $A[p] \neq 0$

$L[i] \leftarrow A[p]$

$i \leftarrow i + 1$

return *L*

Q. Find gcd(31415, 14142) by applying Euclid's algorithm. Estimate how many times it is faster when compared to the algorithm based on consecutive integer checking. (04 Marks)

Iteration	M	N	m%n	gcd(m,n)=gcd(n,m%n)
1	31415	14142	3131	gcd(31415, 14142)=gcd(14142, 3131)
2	14142	3131	1618	gcd(14142, 3141) = gcd(3131, 1618)
3	3131	1618	1513	gcd(3131, 1618) = gcd(1618, 1513)
4	1618	1513	105	gcd(1618, 1513) = gcd(1513, 105)
5	1513	105	43	gcd(1513, 105)= gcd(105, 43)
6	105	43	19	gcd(105, 43)= gcd(43, 19)
7	43	19	5	gcd(43, 19)= gcd(19, 5)
8	19	5	4	gcd(19, 5)= gcd(5, 4)
9	5	4	1	gcd(5, 4)= gcd(4,1)
10	4	1	0	gcd(4, 1)= gcd(1, 0)
11	1	0	gcd(1, 0)= 1	

Gcd(31415, 14142)= 1. Number of divisions = 11

Gcd(31415, 14142) using consecutive integer checking method.

Divide m and n by small(14142). Remainder in both case is not zero. So decrement by 1 and continue division.

Minimum number of division: (if 14142 is the GCD) - 1

Maximum number of division: (if 1 is the GCD) - 14142

So, number of division's will be in between 1 and 14142

So, Euclid's algorithm is faster. To see how much faster

14142/11 = 1285.63≈ 1300

Euclid's algorithm is approximately 1300 times faster than consecutive integer checking method.

Q. Explain the various stages of algorithm design and analysis process using a flow chart. ***(08 M)***

Fundamentals of Algorithmic problem solving

The steps involved in the algorithm design and analysis process are:

Understanding the problem:

The problem given should be understood completely. If the problem is similar to some standard problems and if a known algorithm exists, the same can be used. Otherwise, a new algorithm has to be devised. Creating an algorithm is an art which may never be fully automated. An important step in the design is to specify an instance of the problem.

Ascertain the capabilities of the computational device:

Once the problem is understood, it is important to know the capabilities of the computing device. This can be done by knowing the type of the architecture, speed & memory availability. Appropriate model to be selected from sequential or parallel programming model. Accordingly sequential algorithm or parallel algorithm will be devised.

Exact /approximate solution:

Next step is to choose between solving the problem exactly and solving it approximately. Some problems cannot be solved Examples of problems where an exact solution cannot be solved exactly for most of their instances. Example i) Finding a square root of number. ii) Solutions of non linear equations. Also, the available algorithms for solving a problem exactly can be unacceptably slow because of the problem's intrinsic complexity. Accordingly ***exact algorithm or approximation algorithm*** will be devised.

Decide on the appropriate data structure:

Some algorithms do not demand any ingenuity in representing their inputs. But others are in fact are predicted on ingenious data structures. A data *type* is a well-defined collection of data with a well-defined set of operations on it. A data *structure* is an actual implementation of a particular abstract data type. Algorithms + Datastructure = Programs.

Algorithm design techniques:

- **An algorithm design technique is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.**
- Creating an algorithm is an art which may never be fully automated. By mastering these design strategies, it will become easier for you to devise new and useful algorithms as they provide guidance to design algorithms for new problems.

Methods of specifying an algorithm:

There are mainly two options for specifying an algorithm: use of natural language or pseudocode & Flowcharts.

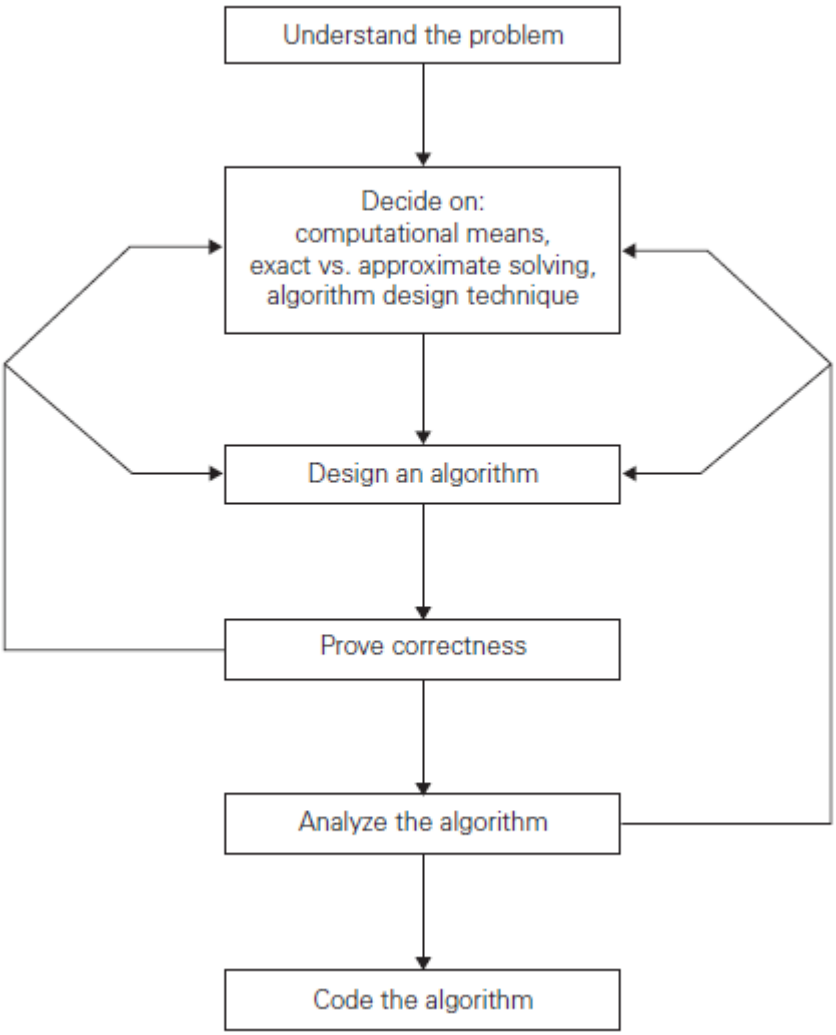
A **Pseudo code** is a mixture of natural language & programming language like constructs. A **flowchart** is a method of expressing an algorithm by a collection of connected geometric shapes.

Proving an algorithm's correctness:

- Once algorithm is devised, it is necessary to show that it computes answer for all the possible legal inputs. This process is called as algorithm validation. The process of validation is to assure us that this algorithm will work correctly independent of issues concerning programming language it will be written in.
- A proof of correctness requires that the solution be stated in two forms. One form is usually as a program which is annotated by a set of assertions about the input and output variables of a program. These assertions are often expressed in the predicate calculus.
- The second form is called a specification, and this may also be expressed in the predicate calculus. A proof consists of showing that these two forms are equivalent in that for every given legal input, they describe same output.
- A complete proof of program correctness requires that each statement of programming language be precisely defined and all basic operations be proved correct. All these details may cause proof to be very much longer than the program.

Analyzing algorithms:

- As an algorithm is executed, it uses the computers central processing unit to perform operation and its memory (both immediate and auxiliary) to hold the program and data. Analysis of algorithms and performance analysis refers to the task of determining how much computing time and storage an algorithm requires.
- This is a challenging area in which some times require great mathematical skill. An important result of this study is that it allows you to make quantitative judgments about the value of one algorithm over another. Another result is that it allows you to predict whether the software will meet any efficiency constraint that exists.



There are two kinds of algorithm efficiency:

Time efficiency: indicates how fast the algorithm runs.

Space efficiency: indicates how much extra memory the algorithm needs

The other desirable characteristics of algorithm are simplicity and generality.

Coding of an algorithm:

Indicates how the objects and operations in the algorithm are represented in the chosen programming language.

Testing

Validating and verification of programs after implementing an algorithm.

Documentation

Generation of reports.

Analysis framework of algorithms

What is basic operation?

The operation that contributes most towards the running time of the algorithm. The statement that executes maximum number of times in a function is also called basic operation. The number of times the basic operation is executed depends on the size of the input. The basic operation is the most time-consuming operation in the algorithm.

Example:

1. A statement present in the innermost loop in the algorithm.
2. Addition operation while adding two matrices.
3. Multiplication operation in matrix multiplication.

Q. **Explain the analysis framework of algorithms.** **(10 M)**

Framework for Analysis

We use a hypothetical model with following assumptions

- Total time taken by the algorithm is given as a function on its input size
- Logical units are identified as one step
- Every step require ONE unit of time
- Total time taken = Total Num. of steps executed

Input's size: Time required by an algorithm is proportional to size of the problem instance. For e.g., more time is required to sort 20 elements than what is required to sort 10 elements. The efficiency is measured as function of algorithm's input size.

Units for Measuring Running Time: Count the number of times an algorithm's basic operation is executed.

How to compute the running time of an algorithm using basic operation.

The running time T(n) is given by:

$$T(n) \sim b*C(n)$$

T= is the running time of the algorithm

n= is the size of the input.

b= execution time for basic operation

c= number of times the basic operation is executed.

Order of growth: (Growth of function)

We expect the algorithms to work faster for all values of n. Some algorithms executes faster for small values of n. But as the value of 'n' increases, they tend to be very slow. So, the behavior of some algorithm changes with increase in value of 'n'.

For example:

1. Suppose T(n) ~ C* C(n) then T(n) varies linearly with increase or decrease in the value of 'n'
2. Suppose T(n) ~ C * C (n^2) the order of growth is quadratic.

The running time of first function is less compared to second. Hence first function is more efficient compared to second function.

The order of growth of a given problem falls into any of the following function (**basic efficiency classes**)

1. Constant (1)
2. Logarithmic (log n)
3. Linear (n)
4. n log n
5. Quadratic (n^2)
6. Cubic (n^3)
7. Exponential (2^n)
8. Factorial (n!)

Constant < Logarithmic < Linear < n log n < Quadratic < Cubic < Exponential < Factorial

Basic Efficiency classes

The time efficiencies of a large number of algorithms fall into the following classes.

fast

slow

1	Constant	
log n	Logarithmic	Binary search
n	Linear	Linear search
n log n	n log n	Quick sort, merge sort, heap sort
n^2	Quadratic	Bubble sort, Selection sort, addition & subtraction of two matrices
n^3	Cubic	Matrix multiplication
2^n	Exponential	Tower of Hanoi
n!	Factorial	Algorithm that generates permutation of sets

High time efficiency

Low time efficiency

Problems:

Q. Compare the order of growth of following functions

1. $n(n+1)$ and $2000n^2$

[Note: Leave the constants and low order terms]

$= n^2 + n$ and $2000n^2$

n^2 and n^2. Both have same order.

Function $n(n+1)$ has same order of growth as that of $2000n^2$

2. $100n^2$ and $0.01n^3$

$100n^2$ = order → n^2

$0.01n^3$= order → n^3

$n^2 < n^3$.

$100n^2$ has lower order of growth than $0.01n^3$

3. $\log_2 n$ and ln n

$\log_2 n = \log_e n / \log_e 2$ $\log_e 2$ – constant

order → log n

ln n = $\log_e n$ order → log n

$\log_2 n$ and ln n has same order of growth

4. $\log_2^2 n$ and $\log_2 n^2$

$\log_2^2 n = \log_2 n * \log_2 n$

$\log_2 n^2 = 2 \log_2 n$

$\log_2^2 n$ has higher order of growth than $\log_2 n^2$

5. 2^{n-1} and 2^n

$2^{n-1} = 2^n / 2$ order - 2^n

2^n = order - 2^n

same order of growth

6. (n-1)! And n!

(n-1)! << n* (n-1)!

(n-1)! has lower order of growth than n!

Worst-Case, Best-Case, and Average-Case Efficiencies

Consider the following algorithm

ALGORITHM *SequentialSearch(*$A[0..n-1]$*, K)*

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n-1]$ and a search key K

//Output: The index of the first element in A that matches K

// or −1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

The running time of this algorithm is different for the same list size n.

The ***worst-case efficiency*** of an algorithm is its efficiency for the worst-case input of size n for which the algorithm runs the longest among all possible inputs of that size. Worst-case analysis provides information about an algorithm's efficiency by bounding its running time from above. It guarantees that for any instance of size n, the running time will not exceed $C_{worst(n)}$, its running time on the worst-case inputs.

Example: For linear search, the worst case occurs when there are no matching elements or the first matching element is the last element of the list. $C_{worst}(n) = n$

The ***best-case efficiency*** of an algorithm is its efficiency for the best-case input of size n for which the algorithm runs the fastest among all possible inputs of that size. Best-case analysis provides information about an algorithm's efficiency by bounding its running time from below. It tells that for any instance of size n, the running time cannot be less than $C_{best(n)}$.

Example: For linear search, the best case occurs when the search key matches the first element in the list. $C_{best}(n) = 1$

The ***average-case efficiency***: Average time taken (number of times the basic operation will be executed) to solve all the possible instances (random) of the input**.** It depends on the probability distribution of instances of the problem.

Example: For linear search, the average case occurs when there is a random input.

$C_{avg}(n) = (n+1)/2;$

To analyze the algorithm's average case efficiency, we must make some assumptions about possible inputs of size n.

The assumptions to linear search are that

(a) the probability of a successful search is equal to p $(0 \leq p \leq 1)$ *and*

(b) *The* probability of the first match occurring in the i^{th} *position of the list is the same* for every *i.*

We can find the average number of key comparisons Cavg(n) as follows.

In the case of a successful search, the probability of the first match occurring in the i^{th} position of the list is p/n for every i, and the number of comparisons made by the algorithm in such a situation is obviously i. In the case of an unsuccessful search, the number of comparisons will be n with the probability of such a search being (1− p).

$$C_{avg}(n) = [1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + i \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}] + n \cdot (1 - p)$$
$$= \frac{p}{n}[1 + 2 + \cdots + i + \cdots + n] + n(1 - p)$$
$$= \frac{p}{n}\frac{n(n+1)}{2} + n(1 - p) = \frac{p(n+1)}{2} + n(1 - p).$$

If p = 1 (the search must be successful), the average number of key comparisons made by sequential search is (n + 1)/2; that is, the algorithm will inspect, on average, about half of the list's elements.

If p = 0 (the search must be unsuccessful), the average number of key comparisons will be n because the algorithm will inspect all n elements on all such inputs.

[NOTE: Average case efficiency is not the average of worst and best case]

Q. Explain asymtotic notations with examples. (6 M)

Asymptotic Notations

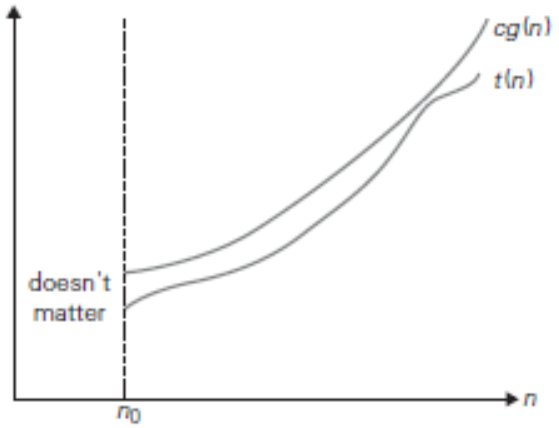
Asymptotic notation is a way of comparing functions that ignores constant factors and small input sizes. Three notations used to compare orders of growth of an algorithm's basic operation count are: **O, Ω, Θ notations**

Big Oh- O notation

Definition: A function t(n) is said to be in O(g(n)), denoted by ***t(n) ∈ O(g(n))***, if t(n) is bounded above by some constant multiple of g(n) for all large n, i.e., if there exist some positive constant c and some nonnegative integer n0 such that

t(n) ≤ cg(n) for all n ≥ n_0

Example: $n \in O(n^2)$, $100n + 5 \in O(n^2)$, $\frac{1}{2} n(n-1) \in O(n^2)$, $n^3 \notin O(n^2)$



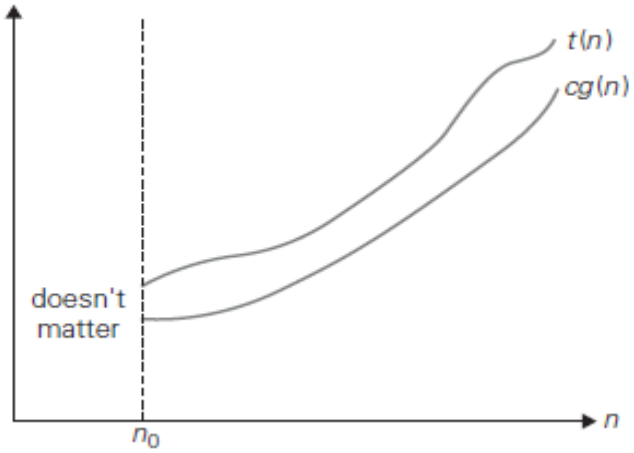
Big-oh notation: $t(n) \in O(g(n))$.

Big Omega- Ω notation

Definition: A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $\boldsymbol{t(n) \in \Omega(g(n))}$, if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n, i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$$\boldsymbol{t(n) \geq cg(n) \quad \text{for all } n \geq n_0.}$$

Example: $n^3 \in \Omega(n^2)$, $1/2\, n(n-1) \in \Omega(n^2)$.



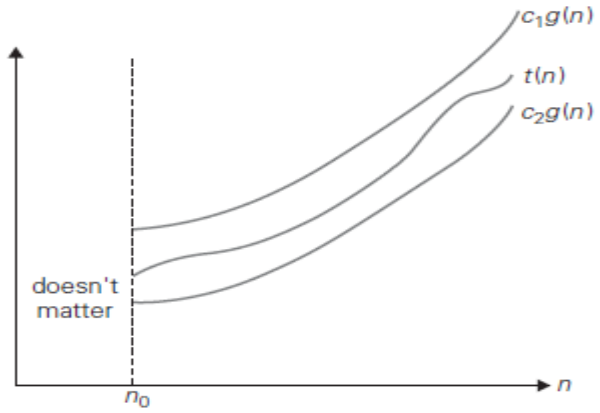
Big-omega notation: $t(n) \in \Omega(g(n))$.

Big Theta- Θ notation

Definition: A function *t (n)* is said to be in θ*(g(n)),* denoted ***t (n) ∈θ (g(n))****,* if *t (n)* is bounded both above and below by some positive constant multiples of *g(n)* for all large *n,* i.e., if there exist some positive constant c_1 *and* c_2 and some nonnegative integer n_0 such that

$c_2 g(n) \le t(n) \le c_1 g(n)$ **for all** $n \ge n_0$.

Example: 1/2 $n(n-1) \in \theta(n^2)$.



Big-theta notation: t (n) ∈θ (g(n)).

Problem 1: Let f(n)= 100n +5. Express f(n) using big-oh

100n+5 – Replace 5 with n, we get 100n +n – c.g(n)

c.g(n) = 100n + n for n=5

= 101n for n=5

By definition of big-oh

$f(n) \le cg(n)$ ***for all*** $n \ge n_0$

100n +5 ≤ 101n for n ≥ 5

So, c=101, g(n) = n and n_0=5. So by definition

f(n) ε O(g(n)) i.e, **100n +5 ε O(n)**

Problem 2: Let f(n)= 100n +5. Express f(n) using big-omega

By definition $f(n) \ge c\ g(n)$ ***for all*** $n \ge n_0$.

100n +5 ≥ 100 n for n ≥ 0

So, c=100, g(n)=n and n_0 =0. So by definition f(n) ε Ω (g(n)) i.e, **100n + 5 ε Ω(n)**

Problem 3: Let f(n)= 100n +5. Express f(n) using big-theta

The constraint to be satisfied is

c_2 $g(n) \leq$ $t(n)$ $\leq$ c_1 $g(n)$ **for all** $n \geq n_0$.

100 n $\leq$ **100n +5** $\leq$ ***101*** ***n*** ***for all n*** $\geq$ **5**

So, c_1= 105 and c_2 = 100 n_o=5 g(n)= n. So by definition **100n +5 ε Θ (n)**

Problem 4: Let f(n)= $10n^3$ +8. Express f(n) using big-oh

Replace 8 by n^3.

The constraint to be satisfied is

f(n) $\leq$ ***c*** ***g(n)*** ***for all*** $n \geq n_0$

$10n^3$ +8 $\leq$ ***11*** ***n^3*** $n \geq 2$

c=11, g(n)= n^3. n_o=2. By definition **$10n^3$ +8 ε O(n^3)**

Problem 5: Let f(n)= $10n^3$ +8. Express f(n) using big-omega

$f(n) \geq c\ g(n)$ ***for all*** $n \geq n_0$.

$10n^3$ +8 $\geq$ **$10n^3$** *for* $n \geq 0$

c= 10, g(n) = n^3 n_0=0. So by definition **$10n^3$ +8 ε Ω(n^3)**

Problem 6: Let f(n)= $10n^3$ +8. Express f(n) using big-theta

The constraint to be satisfied is

c_2 $g(n) \leq$ $t(n)$ $\leq$ c_1 $g(n)$ **for all** $n \geq n_0$.

10 n^3 $\leq$ **$10n^3$ +8** $\leq$ **11** n^3 $n \geq 2$

So, c_1 = 11, c_2 = 10, n_0=2, g(n) = n^3. So by definition **$10n^3$ +8 ε Θ(n^3)**

Problem 7: Let f(n)= 6 * 2^n + n^2. Express f(n) using big-oh

Replace n^2 by 2^n. So c.g(n) = 7 * 2^n for n ≥ 0

So, c=7, g(n) = 2^n, n_0 = 0. So, by definition **6 * 2^n + n^2ε O(2^n)**

Problem 8: Let f(n)= 6 * 2^n + n^2. Express f(n) using big-omega

The constraint to be satisfied is

$f(n) \geq c\ g(n)$ ***for all*** $n \geq n_0$.

6 * 2^n + n^2 $\geq$ **6 * 2^n** $n \geq 0$

So, c=6, g(n) = 2^n, n_0 = 0. So, by definition **6 * 2^n + n^2ε Ω(2^n)**

Problem 9: Let f(n)= 6 * 2^n + n^2. Express f(n) using big-theta

The constraint to be satisfied is

c_2 $g(n) \leq$ $t(n)$ $\leq$ c_1 $g(n)$ **for all** $n \geq n_0$.

6 **2^n** $\leq$ **6 * 2^n + n^2** $\leq$ **7** **2^n** **n ≥ 0**

So, c_2 = 6, g(n)= 2^n , c_1=7 n_0=0. By definition **6 * 2^n + n^2 ε Θ (2^n)**

Useful Property Involving the Asymptotic Notations ***(05 M)***

Q*. Prove that a. If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, prove that $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$

PROOF: Since $t_1(n) \in O(g_1(n))$, there exist some positive constant c_1 and some nonnegative integer n_1 such that

$t_1(n) \le c_1 g_1(n)$ for all $n \ge n_1$.---------- Eq. 1

Similarly, since $t_2(n) \in O(g_2(n))$,

$t_2(n) \le c_2 g_2(n)$ for all $n \ge n_2$.---------- Eq. 2

Let us denote $c_3 = \max\{c_1, c_2\}$ and consider $n \ge \max\{n_1, n_2\}$.

Adding Eq. 1 & Eq. 2

$t_1(n) + t_2(n) \le c_1 g_1(n) + c_2 g_2(n)$

$\le c_3 g_1(n) + c_3 g_2(n) = c_3[g_1(n) + g_2(n)]$

$\le c_3\, 2 \max\{g_1(n), g_2(n)\}$. [Because, if there are four real numbers a_1, b_1, a_2, b_2: if $a_1 \le b_1$ and $a_2 \le b_2$, then $a_1 + a_2 \le 2 \max\{b_1, b_2\}$.]

Hence, $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$, with the constants c and n_0 required by the O definition being $2c_3 = 2 \max\{c_1, c_2\}$ and $\max\{n_1, n_2\}$, respectively.

Using Limits for Comparing Orders of Growth

The order of growth of 2 functions can

be obtained by computing the limit of the ratio of the two functions as shown below:

$$\lim_{n\to\infty}\left(\frac{t(n)}{g(n)}\right) = \begin{cases} 0 & \text{implies that } t(n) \text{ has a smaller order of growth than } g(n) \\ c & \text{implies that } t(n) \text{ has the same order of growth as } g(n) \\ \infty & \text{implies that } t(n) \text{ has a larger order of growth than } g(n) \end{cases}$$

The first two cases mean that $t(n) \in O(g(n))$, the last two mean that $t(n) \in \Omega(g(n))$, and the second case means that $t(n) \in \theta(g(n))$.

L'Hôpital's rule

$$\lim_{n\to\infty} \frac{t(n)}{g(n)} = \lim_{n\to\infty} \frac{t'(n)}{g'(n)}$$

and Stirling's formula

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \quad \text{for large values of } n.$$

EXAMPLE 1 Compare the orders of growth of $\frac{1}{2}n(n-1)$ and n^2.

$$\lim_{n\to\infty}\frac{\frac{1}{2}n(n-1)}{n^2}=\frac{1}{2}\lim_{n\to\infty}\frac{n^2-n}{n^2}=\frac{1}{2}\lim_{n\to\infty}(1-\frac{1}{n})=\frac{1}{2}.$$

Since the limit is equal to a positive constant, the functions have the same order of growth or, symbolically, $\frac{1}{2}n(n-1)\in\Theta(n^2)$. ■

EXAMPLE 2 Compare the orders of growth of $\log_2 n$ and $\sqrt{n}$.

$$\lim_{n\to\infty}\frac{\log_2 n}{\sqrt{n}}=\lim_{n\to\infty}\frac{(\log_2 n)'}{(\sqrt{n})'}=\lim_{n\to\infty}\frac{(\log_2 e)\frac{1}{n}}{\frac{1}{2\sqrt{n}}}=2\log_2 e\lim_{n\to\infty}\frac{1}{\sqrt{n}}=0.$$

Since the limit is equal to zero, log2 n has a smaller order of growth than √n.

EXAMPLE 3 Compare the orders of growth of $n!$ and 2^n.

$$\lim_{n\to\infty}\frac{n!}{2^n}=\lim_{n\to\infty}\frac{\sqrt{2\pi n}\,(\frac{n}{e})^n}{2^n}=\lim_{n\to\infty}\sqrt{2\pi n}\frac{n^n}{2^n e^n}=\lim_{n\to\infty}\sqrt{2\pi n}\left(\frac{n}{2e}\right)^n=\infty.$$

Mathematical Analysis of Nonrecursive Algorithms

General Plan for Analyzing the Time Efficiency of Nonrecursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.

2. Identify the algorithm's basic operation. (As a rule, it is located in the innermost loop.)

3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case efficiencies have to be investigated separately.

4. Set up a sum expressing the number of times the algorithm's basic operation is executed.

5. Using standard formulas and rules of sum manipulation either find a closed form formula for the count or, at the very least, establish its order of growth.

Example:

ALGORITHM *MaxElement(A*[0..*n* − 1]*)*

//Determines the value of the largest element in a given array

//Input: An array *A*[0..*n* − 1] of real numbers

//Output: The value of the largest element in A

maxval ←$A[0]$

for i ←1 **to** $n-1$ **do**

if $A[i]>maxval$

maxval←$A[i]$

return *maxval*

Input size: number of elements in an array- n

Basic operation: Comparison ($A[i]>maxval$)

Total number of times the basic operation is executed depends only on input size 'n'. Therefore no worst, best and average case.

So, total number of times basic operation is executed can be calculated as shown below:

Let us denote C(n), the number of times the comparison statement is executed and express it as a function of size 'n'.

for i ←1 **to** $n-1$ **do**

if $A[i]>maxval$

maxval←$A[i]$

$$C(n) = \sum_{i=1}^{n-1} 1.$$

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n).$$

Write an algorithm to check whether all the elements in a given array of *n* elements are distinct or not. Analyze its efficiency

ALGORITHM *UniqueElements(A*[0..*n* − 1]*)*

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n-1]$

//Output: Returns "true" if all the elements in A are distinct and "false" otherwise

for i ←0 to $n-2$ do

for j ←$i+1$ to $n-1$ do

if $A[i]= A[j]$ return false

return true

Analysis:

Step 1: Input size- n (number of elements)

Step 2: Basic operation statement :**if** ***A[i]= A[j]***

Basic operation count does not depend only on input size(n), but also on whether there are equal elements in the array and if it is, then in which position.

So, worst, best and average case has to be found out separately.

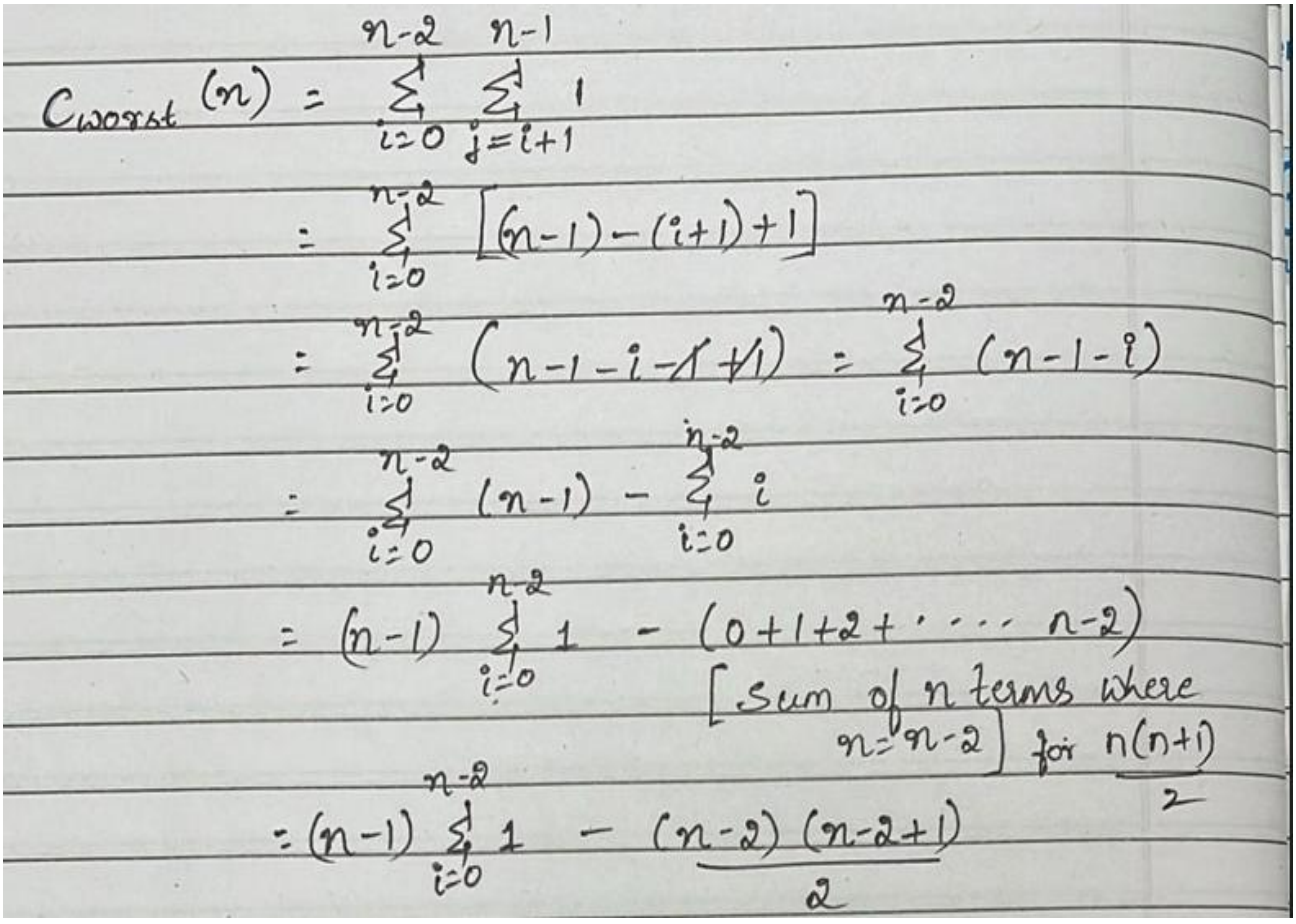
Worst Case

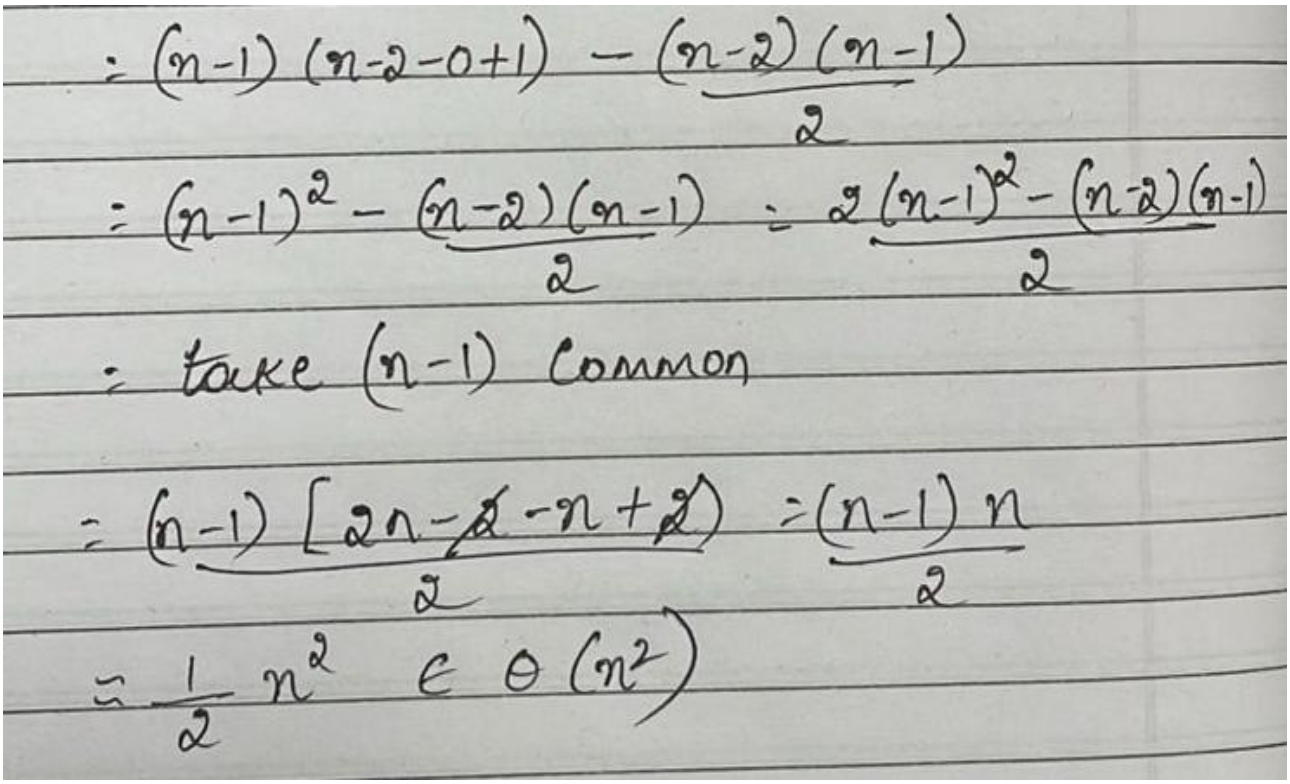
1. Arrays with no equal elements
2. Arrays in which the last two elements are the only pair of equal elements

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i)$$

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2}$$

$$= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2).$$





Write an algorithm to perform multiplication of two matrices and analyze its efficiency.

ALGORITHM *MatrixMultiplication(A*[0..*n* − 1, 0..*n* − 1], *B*[0..*n* − 1, 0..*n* − 1]*)*
//Multiplies two square matrices of order *n* by the definition-based algorithm
//Input: Two *n* ×*n* matrices *A* and *B*
//Output: Matrix *C* = *AB*
for *i* ←0 **to** *n* − 1 **do**
for *j* ←0 **to** *n* − 1 **do**
    C[*i, j*]←0.0
for *k*←0 **to** *n* − 1 **do**
    C[*i, j*]←*C*[*i, j*]+ *A*[*i, k*] ∗*B*[*k, j*]
return *C*

Analysis:
Input size: matrix order- n
Basic operation: multiplication
Total number of times the basic operation is executed depends only on input size 'n'. Therefore no worst, best and average case.
So, total number of times basic operation is executed can be calculated as shown below:

$$M(n) = \sum_{i=0}^{n-1}\sum_{j=0}^{n-1}\sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1}\sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3.$$

Mathematical Analysis of Recursive Algorithms

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.

2. Identify the algorithm's basic operation.

3. Check whether the number of times the basic operation is executed can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case efficiencies must be investigated separately.

4. Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.

5. Solve the recurrence or, at least, ascertain the order of growth of its solution.

Q. Explain the mathematical analysis of recursive algorithm that finds factorial of a number. (6 M)

Example: **ALGORITHM** ***F(n)***

//Computes *n*! recursively

//Input: A non negative integer *n*

//Output: The value of *n*!

if *n* = 0 **return** 1

else return *F(n* − 1*)* * *n*

Analysis:

Input size: Given number n

Basic operation: multiplication

No best, worst, average cases.

Let M(n) denotes the number of multiplications

M(n) = M(n-1) +1 for n>0

M(0) =0 initial condition n=0

Solve the recurrence by using backward substitution method

M(n) = M(n-1) +1

= [M(n-2) + 1] +1

= M (n-2) +2

….

M(i) = M(n-i) + i

…….

M(n)= M(n-n) + n

= M(0) + n Since M(0) = 0

= n

M(n) ε θ(n)

Q. Design a recursive algorithm for solving tower of hanoi problem and give the general plan of analyzing that algorithm. Show that the time complexity of tower of hanoi algorithm is exponential in nature. (8 M)

Algorithm TowerofHanoi(n, source, temp, dest)

//To move n disk from source to destination

//Input: n: total number of disks to be moved

//Output: n disks at destination

if(n=0)

return

TowerofHanoi(n-1, source, dest, temp)

Move n^{th} disk from source to dest

TowerofHanoi(n-1, temp, source, dest)

Analysis for Tower of Hanoi

Input size: The number of disks *n*

Basic operation: moving one disk

The number of moves *M(n)* depends only on 'n'. No worst, best, average case efficiency.

It can be expressed by the following recurrence relation

$$M(n)= \begin{cases} 1 & n = 1 \\ M(n-1) + 1 + M(n-1) & \text{for } n > 1. \end{cases}$$

$$M(n)= \begin{cases} 1 & n = 1 \\ 2M(n-1) + 1 & \text{for } n > 1. \end{cases}$$

We solve this recurrence by the method of backward substitutions:

$M(n) = 2M(n-1) + 1$ substitute $M(n-1) = 2M(n-2) + 1$

$= 2[2M(n-2) + 1] + 1$

$= 2^2M(n-2) + 2 + 1$ substitute $M(n-2) = 2M(n-3) + 1$

$= 2^2[2M(n-3) + 1] + 2 + 1$

$= 2^3M(n-3) + 2^2 + 2 + 1.$

$= 2^4M(n - 4) + 2^3 + 2^2 + 2 + 1$

generally, after i substitutions, we get

$M(n) = 2^iM(n - i) + 2^{i-1} + 2^{i-2} + \ldots + 2 + 1$ $2^{i-1} + 2^{i-2} + \ldots + 2 + 2^0$ *(Total n terms) n=i*

$= 2^iM(n - i) + 2^i - 1.$ $S=a(r^n-1)/r-1$

$=1(2^i-1)/2-1$

$= 2^i-1$

Since the initial condition is specified for $n = 1$, which is achieved for $i = n - 1$, we get the following formula for the solution to recurrence

$M(n) = 2^{n-1}M(n - (n - 1)) + 2^{n-1} - 1$

$F= 2^{n-1}M(1) + 2^{n-1} - 1$

$= 2^{n-1} + 2^{n-1} - 1$

$= 2. 2^{n-1} - 1$

$= 2^n - 1.$

Time complexity= $\theta(2^n)$ which is exponential.

Time Complexity

Statement	**S/e**	**Frequency**	**Total steps**
Algorithm Sum(a, n)	0	-	**Θ(0)**
{	0	-	**Θ(0)**
s:=0.0;	1	1	**Θ(1)**
for i:=1 to n do	1	n+1	**Θ(n+1)**
s:=s + a[i];	1	n	**Θ(n)**
return s;	1	1	**Θ(1)**
}	0	-	**Θ(0)**
Total			**Θ(n)**

Statement	**S/e**	**Frequency**		**Total steps**	
		n=0	**n>0**	**n=0**	**n>0**
Algorithm RSum(a, n)	0	-	-	**Θ(0)**	**Θ(0)**
{	0	-	-	**Θ(0)**	**Θ(0)**
if(n≤0) then	1	1	1	**Θ(1)**	**Θ(1)**
return 0.0;	1	1	0	**Θ(1)**	**Θ(0)**
else return					
RSum(a,n-1)+a[n];	1+x	0	1	**Θ(0)**	**Θ(1+x)**
}	0	-	-	**Θ(0)**	**Θ(0)**
Total				**2**	**Θ(1+x)**
					$x=t_{Rsum}(n-1)$

Statement	S/e	Frequency	Total steps
Algorithm Add(a, b, c, m, n)	0	-	**Θ(0)**
{	0	-	**Θ(0)**
for i:=1 to m do	1	m+1	**Θ(m)**
for j:= 1 to n do	1	m(n+1)	**Θ(mn)**
c[i,j]:= a[i, j] + b[i, j];	1	mn	**Θ(mn)**
}	0	-	**Θ(0)**
Total			**Θ(mn)**

1. Algorithm X(int N)

```
{
  int P;  P=N;
  for i ←1 to N
  {
     Printf("\n%d\t * \t %d= %d", N, i, P);
   P=P + N;
  }  }
```

i) What does this algorithm compute?
ii) What is the basic operation?
iii) How many times the basic operation is executed?
iv) What is the efficiency class of this algorithm?

Soln: i) computes Table of N till N*N
ii) N*i
iii) N times
iv) Θ(n)- linear

2. Consider the following algorithm

```
Algorithm Mystery (n)
        //input: A non negative integer n
  S ←0
  for i←1 to n do
        S ← S + i * i;
  return S
```

i) What does this algorithm compute?
ii) What is its basic operation?
iii) How many times is the basic operation executed?
iv) What is the efficiency class of this algorithm?

Soln: i)sum of squares of n
ii) S + i * i;
iii) N times
iv) Θ(n)- linear

Brute Force

Introduction: Brute force is a straightforward approach to problem solving, usually directly based on the problem's statement and definitions of the concepts involved. Example algorithms include:

- Linear search
- Selection sort
- Bubble sort
- String Matching

Advantages:

1. Useful for solving small-size instances of a problem
2. Applicable to wide variety of problems
3. It yields reasonable algorithms for problems like sorting, searching, string matching etc.
4. No limitation on instance size
5. Simple
6. Can be designed easily

Disadvantages:

1. Brute force algorithms are rarely efficient

Q. Write an algorithm for selection sort and show that the time complexity of this algorithm is quadratic. (08 M)

Q. Give an algorithm for selection sort. If C(n) denotes the number of times the algorithm is executed (n denotes input size), obtain an expression for C(n) (07 M)

Selection Sort

Idea:

- Scan the entire given list to find its smallest element
- Exchange it with the first element.
- Scan the list starting from second element to find the next smallest element.
- Exchange it with the second element.
- Continue until the array is sorted

Generally, on the *i*th pass, the algorithm searches for the smallest item among the last $n - i$ elements and swaps it with Ai :

$$\underbrace{A_0 \le A_1 \le \cdots \le A_{i-1}}_{\text{in their final positions}} \mid \underbrace{A_i, \ldots, A_{min}, \ldots, A_{n-1}}_{\text{the last } n-i \text{ elements}}$$

After $n - 1$ passes, the list is sorted.

ALGORITHM *SelectionSort(A*[0..$n - 1$]*)*

//Sorts a given array by selection sort

//Input: An array *A*[0..$n - 1$] of orderable elements

//Output: Array *A*[0..$n - 1$] sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

min$\leftarrow i$

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if *A*[j]<*A*[*min*] *min*$\leftarrow j$

swap *A*[i] and *A*[*min*]

Example: 89, 45, 68, 90, 29, 34, 17

89	45	68	90	29	34	**17**
17 |	45	68	90	**29**	34	89
17	29 |	68	90	45	**34**	89
17	29	34 |	90	**45**	68	89
17	29	34	45 |	90	**68**	89
17	29	34	45	68 |	90	**89**
17	29	34	45	68	89 |	90

Analysis for selection sort

Analysis for selection sort:

Input size - number of elements n

Basic operation-key comparison A[j]<A[min].

The number of times it is executed depends only on the array size.

The expression for the number of key comparisons C(n) is given by:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i).$$

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2}$$

$$= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2} n^2 \in \Theta(n^2).$$

- Time complexity of selection sort = θ (n^2)
- No. of key swaps= n-1

Problem: Sort the following using selection sort

S, E, L, E, C, T, I, O, N

S	E	L	E	**C**	T	I	O	N
C	**E**	L	E	S	T	I	O	N
C	E	**L**	E	S	T	I	O	N
C	E	E	**L**	S	T	**I**	O	N
C	E	E	I	S	T	**L**	O	N
C	E	E	I	L	T	S	O	**N**
C	E	E	I	L	N	S	**O**	T
C	E	E	I	L	N	O	**S**	T
C	E	E	I	L	N	O	S	T

Selection Sort is **Unstable and Inplace.**

Stable: If input list contains two equal elements in positions i and j where i<j, then in the sorted list they have to be in positions i' and j' such that i' < j'. An algorithm is called stable, if it preserves the relative order of any two equal elements in its input.

Inplace: If the algorithm does not consume extra memory units, it is inplace.

Bubble Sort:

Idea:

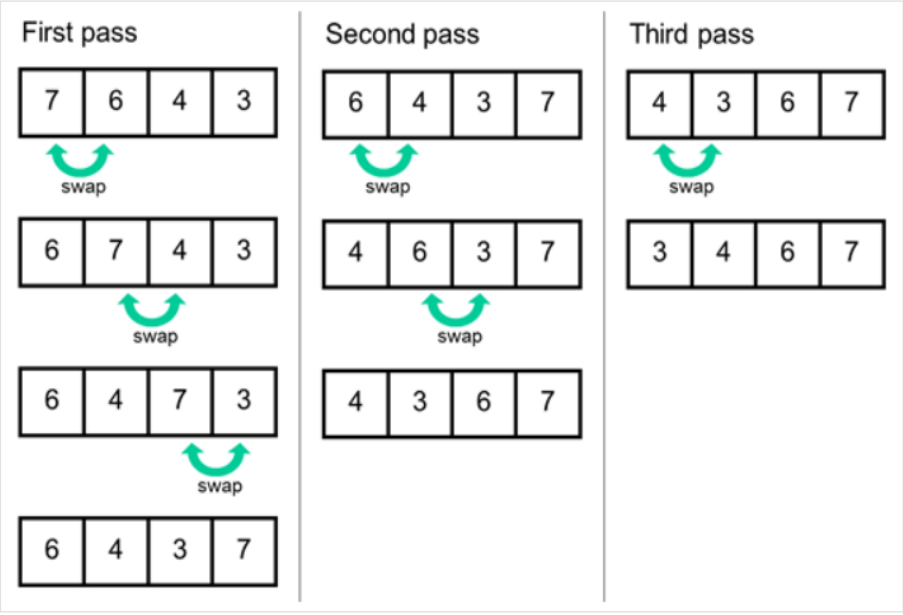
- Repeatedly pass through the array
- Swaps adjacent elements that are out of order
- The largest element bubbles up to the last position in the first pass
- The second largest element bubbles up to its position in the second pass
- The list is sorted after n-1 passes.

i^{th} pass of a bubble sort can be represented by the following figure:

$$A_0, \ldots, A_j \overset{?}{\leftrightarrow} A_{j+1}, \ldots, A_{n-i-1} \mid \underbrace{A_{n-i} \leq \cdots \leq A_{n-1}}_{\text{in their final positions}}$$

Example

89	? ↔	45		68		90		29		34		17					
45		89	? ↔	68		90		29		34		17					
45		68		89	? ↔	90	? ↔	29		34		17					
45		68		89		29		90	? ↔	34		17					
45		68		89		29		34		90	? ↔	17					
45		68		89		29		34		17	|	90					
45	? ↔	68	? ↔	89	? ↔	29		34		17	|	90					
45		68		29		89	? ↔	34		17	|	90					
45		68		29		34		89	? ↔	17	|	90					
45		68		29		34		17	|	89		90					



ALGORITHM ***BubbleSort(A[0..n − 1])***

//Sorts a given array by bubble sort

//Input: An array *A[0..n − 1] of orderable elements*

//Output: Array *A[0..n − 1] sorted in non decreasing order*

for *i ←0 to n − 2 do*

for *j ←0 to n − 2 − i do*

if *A[j + 1]<A[j]*

swap A[j] and A[j + 1]

Analysis for bubble sort

- Input size - number of elements *n*
- Basic operation-key comparison

  A[j +1]<A[j].
- The number of times it is executed can be expressed by the following sum:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2).$$

Bubble Sort is **Stable and Inplace.**